

Vapor: Plataforma de Venda e Distribuição Digital de Jogos

Professor: João Armênio <joao.armenio@ulife.com.br>

Equipe:

- [Osvaldo Santana Neto](#)
- [João Roiko](#)
- [Paulo Eduardo Sitko Duarte](#)

Tema

Desenvolvimento de uma plataforma web de comércio eletrônico voltada à venda, distribuição e gerenciamento de jogos digitais.

Descrição do Projeto

O Vapor é uma aplicação web que permite a compra, venda e download de jogos digitais. Qualquer usuário registrado pode tanto comprar jogos e gerenciar sua biblioteca pessoal quanto publicar seus próprios títulos na plataforma. Administradores são responsáveis pela moderação do catálogo.

O sistema contempla funcionalidades essenciais de uma loja digital: catálogo com busca e filtros, carrinho de compras, processamento de pagamentos, biblioteca pessoal com gerenciamento de downloads e solicitação de reembolso.

Análise de Requisitos

Requisitos Funcionais

RF-01: Cadastro de usuário

O sistema deve permitir o registro de novos usuários com e-mail, senha e nome de exibição.

Prioridade: Essencial

RF-02: Autenticação

O sistema deve permitir *login* e *logout* com credenciais de e-mail e senha.

Prioridade: Essencial

RF-03: Perfis de acesso

O sistema deve distinguir dois perfis: Usuário e Administrador. Qualquer usuário pode comprar e publicar jogos.

Prioridade: Essencial

RF-04: Catálogo de jogos

O sistema deve exibir uma listagem paginada de jogos disponíveis para compra.

Prioridade: Essencial

RF-05: Busca e filtros

O sistema deve permitir busca por nome e filtragem por gênero, preço, plataforma e data de lançamento.

Prioridade: Essencial

RF-06: Página de detalhes do jogo

O sistema deve exibir informações detalhadas de cada jogo: descrição, mídia, requisitos de sistema e preço.

Prioridade: Essencial

RF-07: Carrinho de compras

O sistema deve permitir adicionar, remover e visualizar itens em um carrinho antes da compra.

Prioridade: Essencial

RF-08: Processamento de pagamento

O sistema deve processar pagamentos via cartão de crédito e PIX.

Prioridade: Essencial

RF-10: Biblioteca pessoal

O sistema deve manter uma biblioteca com todos os jogos adquiridos pelo usuário.

Prioridade: Essencial

RF-11: Download de jogos

O sistema deve fornecer links de download para os jogos presentes na biblioteca do usuário.

Prioridade: Essencial

RF-12: Cadastro de jogos

O sistema deve permitir que usuários cadastrem novos jogos com metadados, mídias e arquivos de instalação.

Prioridade: Essencial

RF-13: Gerenciamento de preços

O sistema deve permitir que usuários definam e alterem o preço de seus jogos.

Prioridade: Essencial

RF-14: Painel de vendas

O sistema deve exibir ao usuário publicador métricas de vendas: unidades vendidas e receita.

Prioridade: Importante

RF-15: Reembolso

O sistema deve permitir a solicitação de reembolso dentro de um prazo configurável após a compra.

Prioridade: Desejável

RF-16: Moderação de jogos

O sistema deve fornecer ao administrador uma fila de jogos submetidos para revisão, permitindo aprová-los ou rejeitá-los antes de ficarem disponíveis no catálogo.

Prioridade: Essencial

Requisitos Não Funcionais

RNF-01: Responsividade

A interface deve ser utilizável em dispositivos com largura mínima de 375px (mobile) até desktop.

Categoria: Usabilidade

RNF-02: Tempo de resposta

As páginas do catálogo devem carregar em no máximo 2 segundos sob carga normal.

Categoria: Desempenho

RNF-03: Disponibilidade

O sistema deve ter disponibilidade mínima de 99% (exceto janelas de manutenção programada).

Categoria: Confiabilidade

RNF-04: Segurança de senhas

Senhas devem ser armazenadas com hash seguro (bcrypt ou argon2) e nunca trafegar em texto plano.

Categoria: Segurança

RNF-05: HTTPS obrigatório

Toda comunicação entre cliente e servidor deve ocorrer via HTTPS/TLS.

Categoria: Segurança

RNF-06: Proteção contra fraudes

O módulo de pagamento deve validar dados e implementar proteção contra ataques de replay e CSRF.

Categoria: Segurança

RNF-07: Escalabilidade horizontal

A arquitetura deve permitir escalar horizontalmente os serviços de catálogo e download de forma independente.

Categoria: Escalabilidade

RNF-08: Armazenamento de arquivos

Arquivos de jogos devem ser armazenados em serviço de object storage com redundância geográfica.

Categoria: Infraestrutura

RNF-09: Internacionalização

O sistema deve suportar pelo menos pt-BR e en-US na interface.

Categoria: Usabilidade

RNF-10: Acessibilidade

A interface deve seguir as diretrizes WCAG 2.1 nível AA.

Categoria: Usabilidade

RNF-11: Compatibilidade

O sistema deve funcionar nos navegadores Chrome, Firefox, Safari e Edge em suas duas últimas versões estáveis.

Categoria: Portabilidade

RNF-12: LGPD

O sistema deve estar em conformidade com a Lei Geral de Proteção de Dados (Lei nº 13.709/2018).

Categoria: Legal

Regras de Negócio

RN-01: Condições de reembolso

O reembolso só pode ser solicitado em até 14 dias após a compra e com menos de 2 horas de uso registrado.

RN-02: Split de receita

O publicador recebe 70% do valor líquido de cada venda; os 30% restantes são retidos pela plataforma.

RN-03: Compra única por título

Um jogo já presente na biblioteca do usuário não pode ser comprado novamente.

RN-04: Reembolso ao meio original

Reembolsos aprovados devolvem o valor ao meio de pagamento original.

RN-05: Aprovação de jogos

Jogos cadastrados por usuários ficam com status "Em análise" até aprovação do administrador e só aparecem no catálogo após aprovados.

User Stories

Usuário

US-01: Cadastro

Como usuário, quero me cadastrar na plataforma com meu e-mail e senha, para ter acesso ao catálogo e poder realizar compras e publicar jogos.

Critérios de aceitação:

- Validação de formato de e-mail.
- Senha com no mínimo 8 caracteres, contendo letra e número.
- Confirmação de e-mail via link enviado ao endereço informado.
- Exibição de mensagem de erro caso o e-mail já esteja cadastrado.

US-02: Login

Como usuário, quero fazer login com meu e-mail e senha, para acessar minha conta e biblioteca.

Critérios de aceitação:

- Redirecionamento para a página principal após login bem-sucedido.
- Mensagem de erro genérica para credenciais inválidas (sem revelar se o e-mail existe).
- Bloqueio temporário após 5 tentativas falhas consecutivas.

US-03: Navegar pelo catálogo

Como usuário, quero navegar pelo catálogo de jogos disponíveis, para descobrir novos títulos.

Critérios de aceitação:

- Listagem paginada com 20 jogos por página.
- Exibição de capa, nome e preço de cada jogo.
- Ordenação por relevância, preço e data de lançamento.

US-04: Buscar e filtrar jogos

Como usuário, quero buscar jogos por nome e aplicar filtros, para encontrar rapidamente o que procuro.

Critérios de aceitação:

- Campo de busca com sugestão automática (autocomplete).
- Filtros combinados: gênero, faixa de preço, plataforma (Windows/macOS/Linux).
- Resultados atualizados sem recarregar a página (AJAX/SPA).

US-05: Visualizar detalhes do jogo

Como usuário, quero ver a página de detalhes de um jogo, para decidir se quero comprá-lo.

Critérios de aceitação:

- Exibição de: descrição, screenshots/trailer, gêneros, data de lançamento, publicador, requisitos de sistema e preço.
- Botão "Adicionar ao Carrinho" visível (ou "Na Biblioteca" se já adquirido).

US-06: Adicionar ao carrinho

Como usuário, quero adicionar jogos ao meu carrinho de compras, para adquirir múltiplos títulos em uma única transação.

Critérios de aceitação:

- Feedback visual ao adicionar item.
- Impedimento de adicionar jogo já presente no carrinho ou na biblioteca.
- Contador de itens visível no ícone do carrinho.

US-07: Finalizar compra

Como usuário, quero finalizar a compra dos jogos no meu carrinho, para adicioná-los à minha biblioteca.

Critérios de aceitação:

- Resumo do pedido com itens e valor total.
- Seleção de meio de pagamento (cartão, PIX).
- Confirmação de compra com número do pedido.
- Jogos adicionados automaticamente à biblioteca após pagamento confirmado.

US-08: Acessar a biblioteca

Como usuário, quero acessar minha biblioteca pessoal, para ver e gerenciar os jogos que possuo.

Critérios de aceitação:

- Listagem de todos os jogos adquiridos com capa e nome.
- Busca e ordenação dentro da biblioteca.
- Botão de download em cada título.

US-09: Baixar um jogo

Como usuário, quero baixar um jogo da minha biblioteca, para instalá-lo no meu computador.

Critérios de aceitação:

- Geração de link de download temporário e seguro (URL assinada com expiração).
- Indicação do tamanho do arquivo antes do download.
- Seleção de plataforma quando o jogo tiver múltiplas versões.

US-10: Solicitar reembolso

Como usuário, quero solicitar o reembolso de um jogo, caso ele não atenda às minhas expectativas dentro do prazo permitido.

Critérios de aceitação:

- Validação das condições de elegibilidade (RN-01).
- Campo para motivo do reembolso.
- Reembolso automático ao meio de pagamento original após aprovação.

US-11: Cadastrar um jogo

Como usuário, quero cadastrar um novo jogo na plataforma, para disponibilizá-lo para venda.

Critérios de aceitação:

- Formulário com: título, descrição, gêneros, plataformas suportadas, requisitos de sistema, mídia (screenshots, trailer) e arquivo de instalação.
- Validação de campos obrigatórios e formatos de arquivo.
- Status inicial "Em análise" até aprovação pelo administrador (RN-05).

US-12: Definir e alterar preço

Como usuário, quero definir e alterar o preço do meu jogo, para adequá-lo à estratégia comercial.

Critérios de aceitação:

- Campo de preço em BRL com máscara monetária.
- Opção de marcar o jogo como gratuito (preço R\$ 0,00).
- Histórico de alterações de preço visível.

US-13: Visualizar métricas de vendas

Como usuário, quero visualizar as métricas de vendas dos meus jogos, para acompanhar o desempenho comercial.

Critérios de aceitação:

- Dashboard com: unidades vendidas, receita bruta e receita líquida (após split da plataforma).
- Filtro por jogo e período.
- Exportação de relatório em CSV.

Administrador

US-14: Aprovar ou rejeitar jogos

Como administrador, quero aprovar ou rejeitar jogos cadastrados por usuários, para garantir a qualidade e conformidade do catálogo.

Critérios de aceitação:

- Fila de jogos com status "Em análise".
- Visualização completa dos dados e mídias do jogo.
- Opção de aprovar, rejeitar (com motivo) ou solicitar ajustes ao usuário.

Diagramas de Caso de Uso

Visão Geral

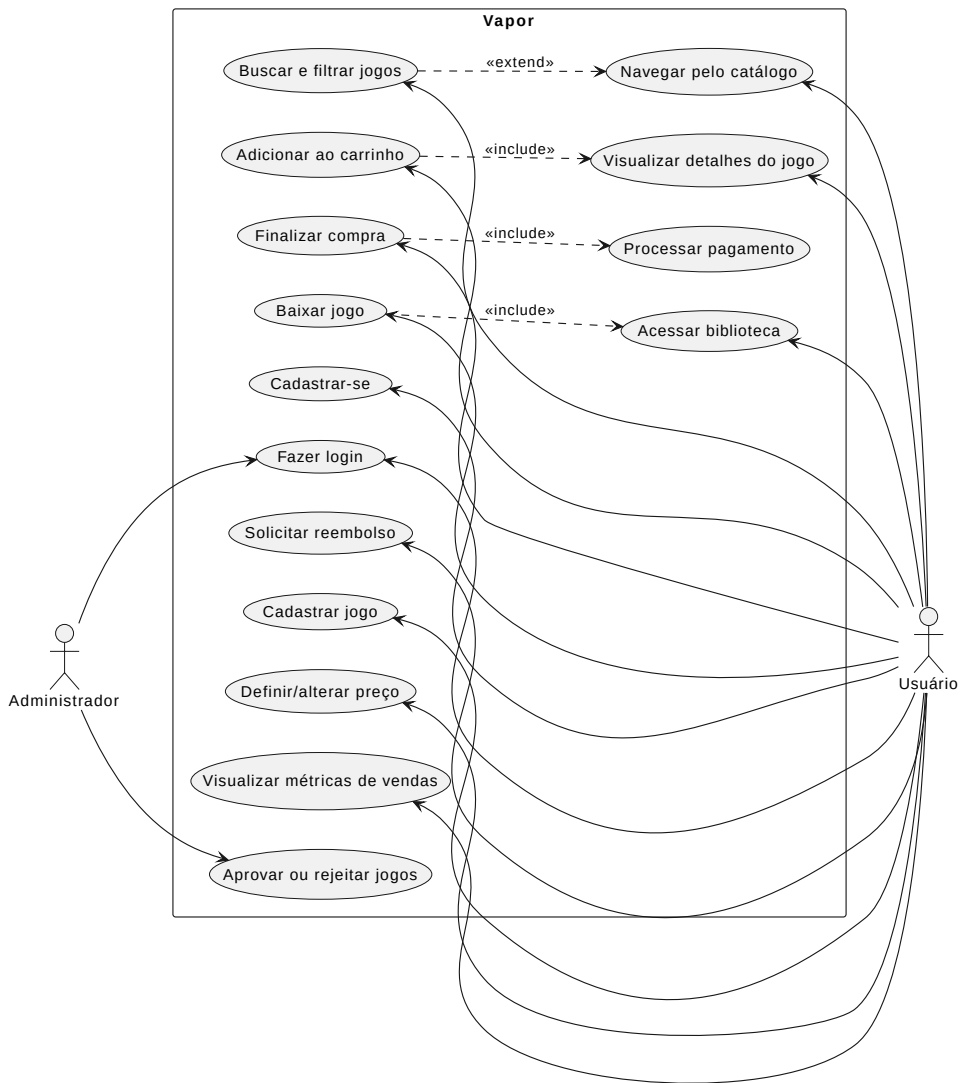


Figura 1. Diagrama de Caso de Uso – Visão Geral

Usuário

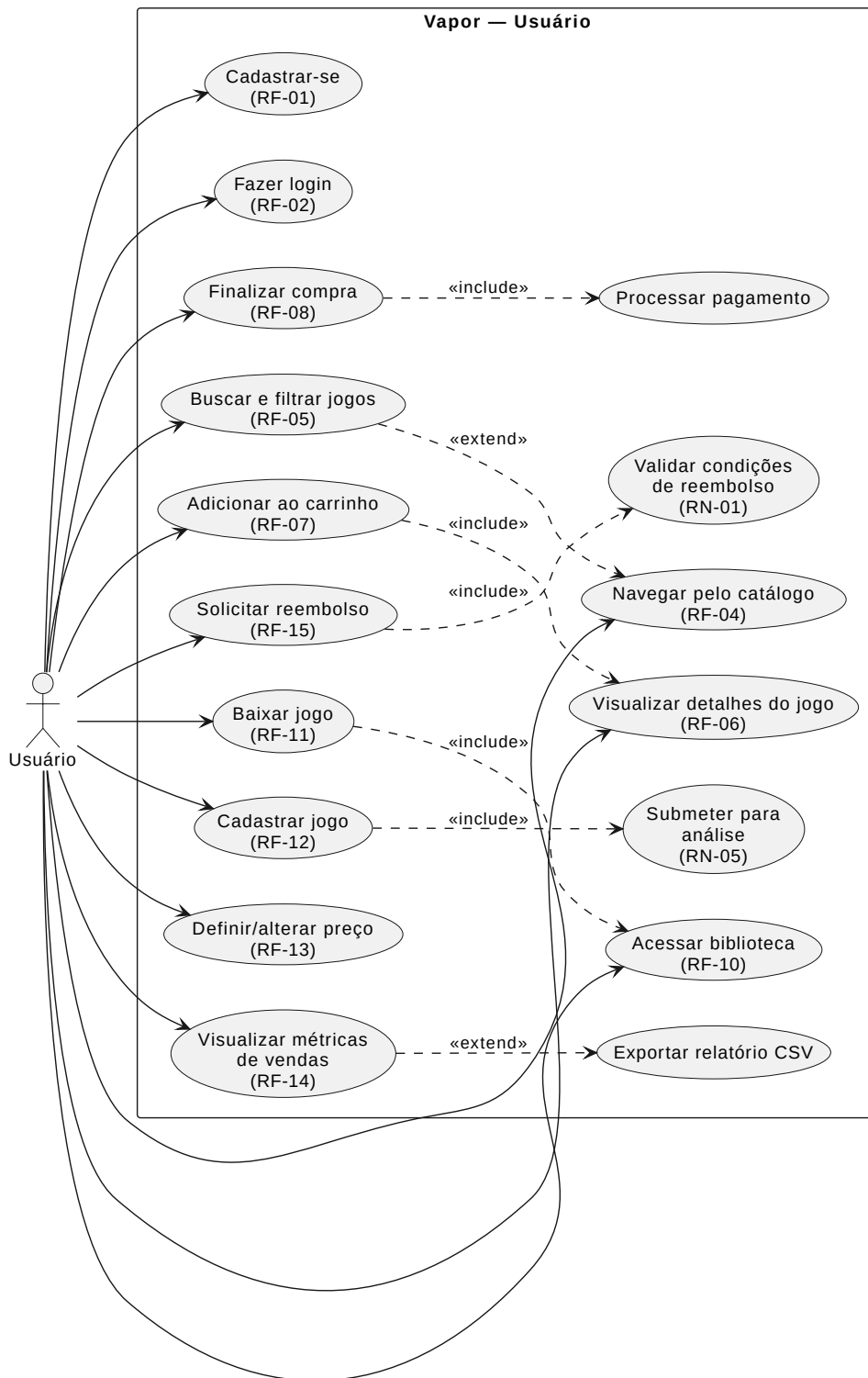


Figura 2. Diagrama de Caso de Uso — Usuário

Administrador

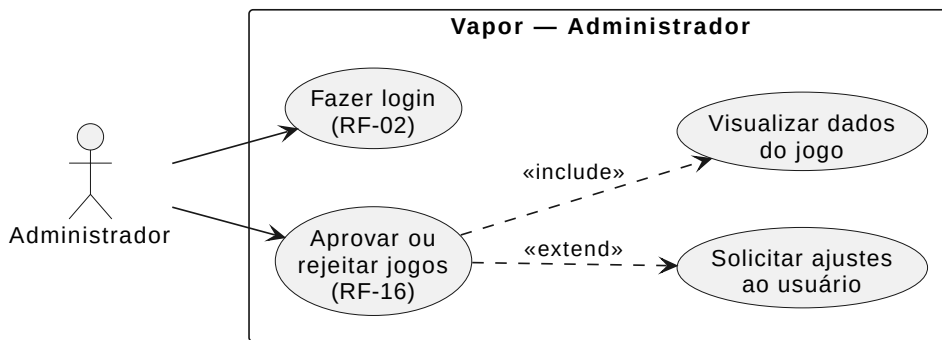


Figura 3. Diagrama de Caso de Uso – Administrador

Casos de Uso

UC-07: Finalizar compra

Ator

Usuário

Pré-condições

Usuário autenticado; carrinho com ao menos um item.

Fluxo principal

1. Usuário acessa o carrinho e inicia o checkout.
2. Sistema exibe resumo do pedido com itens e valor total.
3. Usuário seleciona o meio de pagamento (cartão de crédito ou PIX).
4. Usuário confirma a compra.
5. Sistema processa o pagamento junto ao provedor externo.
6. Sistema adiciona os jogos à biblioteca do usuário.
7. Sistema exibe confirmação com número do pedido.

Exceções

- Pagamento recusado: sistema notifica o usuário e mantém o carrinho intacto para nova tentativa.

Pós-condições

Pedido registrado no histórico; jogos disponíveis na biblioteca do usuário.

UC-10: Solicitar reembolso

Ator

Usuário

Pré-condições

Usuário autenticado; jogo presente na biblioteca; condições de elegibilidade atendidas (RN-01).

Fluxo principal

1. Usuário acessa o histórico de pedidos e seleciona o jogo a reembolsar.
2. Usuário inicia a solicitação e informa o motivo.
3. Sistema valida as condições de elegibilidade (prazo e tempo de uso).
4. Sistema aprova o reembolso e devolve o valor ao meio de pagamento original.
5. Sistema remove o jogo da biblioteca do usuário.

Exceções

- Condições não atendidas (prazo expirado ou tempo de uso excedido): sistema rejeita a solicitação e exibe o motivo ao usuário.

Pós-condições

Valor reembolsado ao meio de pagamento original; jogo removido da biblioteca.

UC-11: Cadastrar jogo

Ator

Usuário

Pré-condições

Usuário autenticado.

Fluxo principal

1. Usuário acessa o painel e inicia o cadastro de novo jogo.
2. Usuário preenche os metadados: título, descrição, gêneros, plataformas suportadas e requisitos de sistema.
3. Usuário faz upload de mídia (screenshots e trailer) e do arquivo de instalação.
4. Usuário define o preço e submete o jogo para análise.
5. Sistema registra o jogo com status "Em análise" (RN-05) e notifica o administrador.

Exceções

- Campos obrigatórios ausentes ou formato de arquivo inválido: sistema exibe os erros e aguarda correção antes de prosseguir.

Pós-condições

Jogo registrado com status "Em análise", visível apenas para o publicador e administradores.

UC-14: Aprovar ou rejeitar jogo

Ator

Administrador

Pré-condições

Usuário autenticado com perfil Administrador; ao menos um jogo com status "Em análise" na

fila.

Fluxo principal

1. Administrador acessa a fila de jogos em análise.
2. Administrador revisa os metadados, mídia e arquivo de instalação do jogo.
3. Administrador aprova o jogo.
4. Sistema altera o status para "Aprovado", publica o jogo no catálogo e notifica o publicador.

Exceções

- Rejeição: administrador informa o motivo; sistema altera o status para "Rejeitado" e notifica o publicador.
- Solicitação de ajustes: administrador descreve os ajustes necessários; sistema mantém o status "Em análise" e notifica o publicador.

Pós-condições

Jogo publicado no catálogo (aprovação) ou publicador notificado com o motivo da recusa (rejeição).

Diagramas de Atividades

UC-07: Finalizar Compra

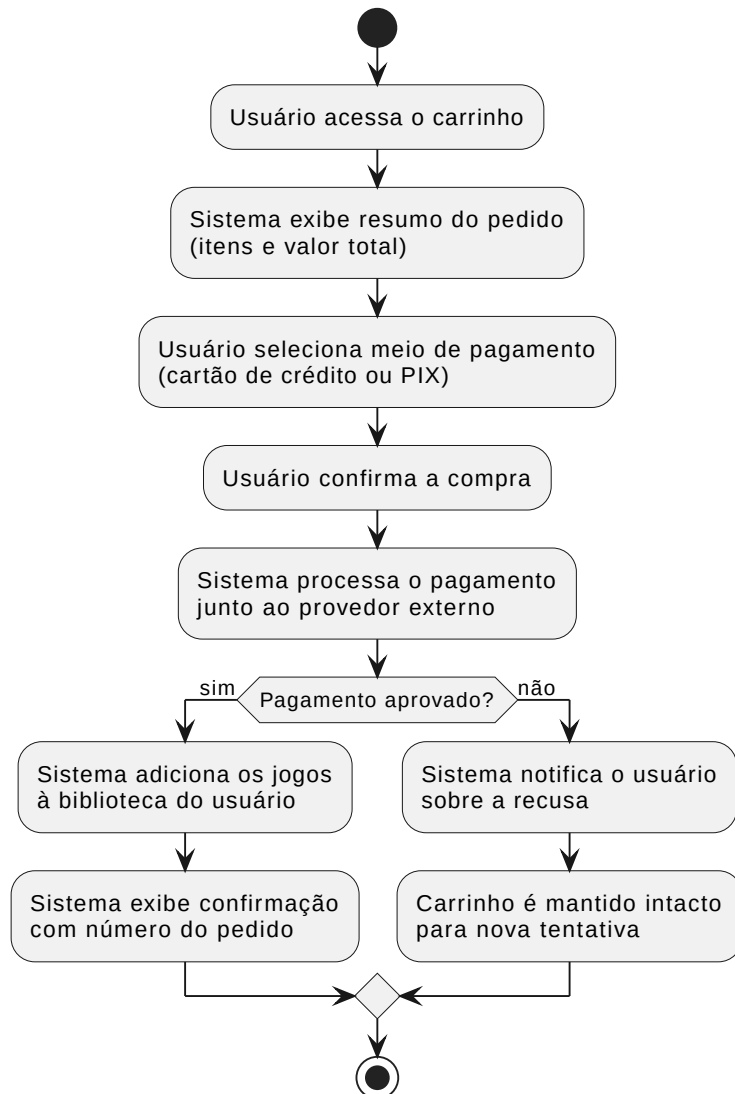


Figura 4. Diagrama de Atividades – UC-07: Finalizar Compra

UC-10: Solicitar Reembolso

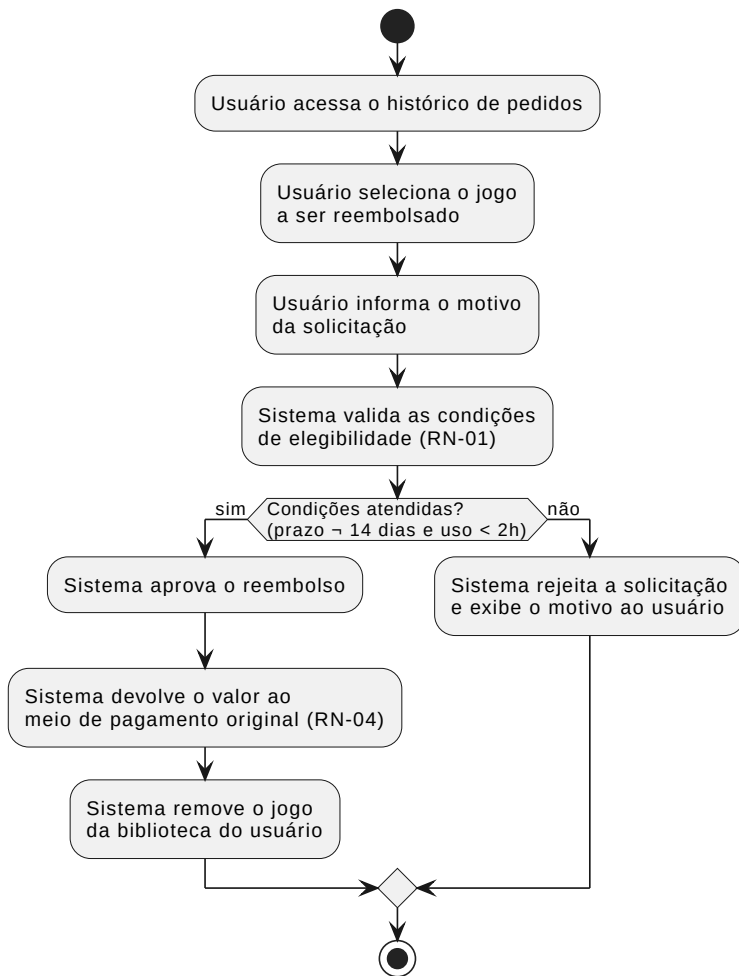


Figura 5. Diagrama de Atividades – UC-10: Solicitar Reembolso

UC-11: Cadastrar Jogo

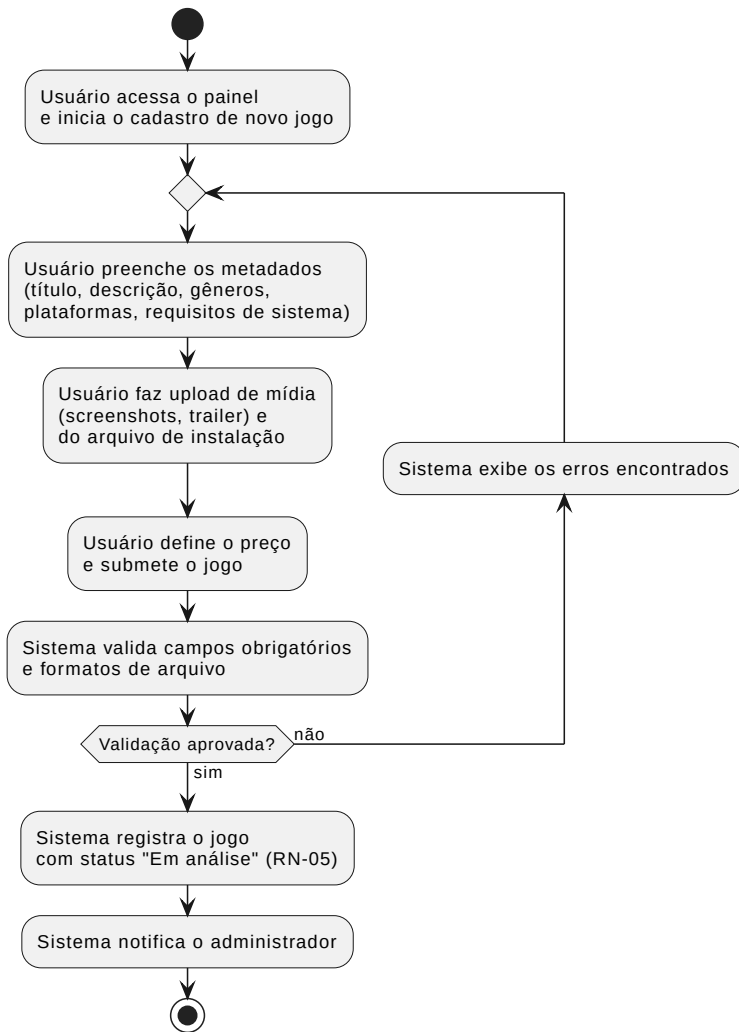


Figura 6. Diagrama de Atividades – UC-11: Cadastrar Jogo

UC-14: Aprovar ou Rejeitar Jogo

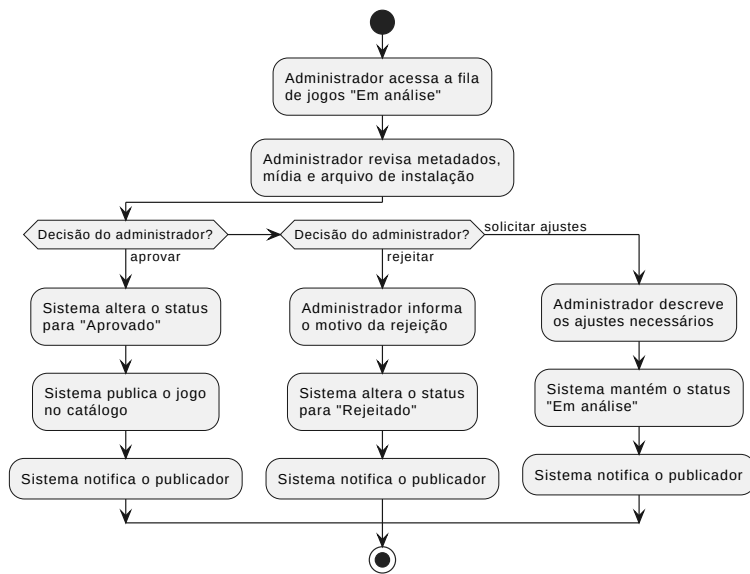


Figura 7. Diagrama de Atividades – UC-14: Aprovar ou Rejeitar Jogo

Entidades Principais

As entidades abaixo servem de referência para os diagramas:

- User (id, name, email, password_hash, role, created_at)
- Game (id, title, description, creator_id, price, release_date, status, created_at)
- Genre (id, name)
- GameGenre (game_id, genre_id)
- Platform (id, name)
- GamePlatform (game_id, platform_id, download_url, file_size)
- CartItem (id, user_id, game_id, added_at)
- Order (id, user_id, total, payment_method, status, created_at)
- OrderItem (id, order_id, game_id, price_paid)
- Library (id, user_id, game_id, acquired_at)
- Refund (id, order_item_id, reason, status, created_at)

Modelo Conceitual do Banco de Dados

O modelo conceitual apresenta as entidades do sistema e seus relacionamentos em alto nível, sem detalhes de implementação.

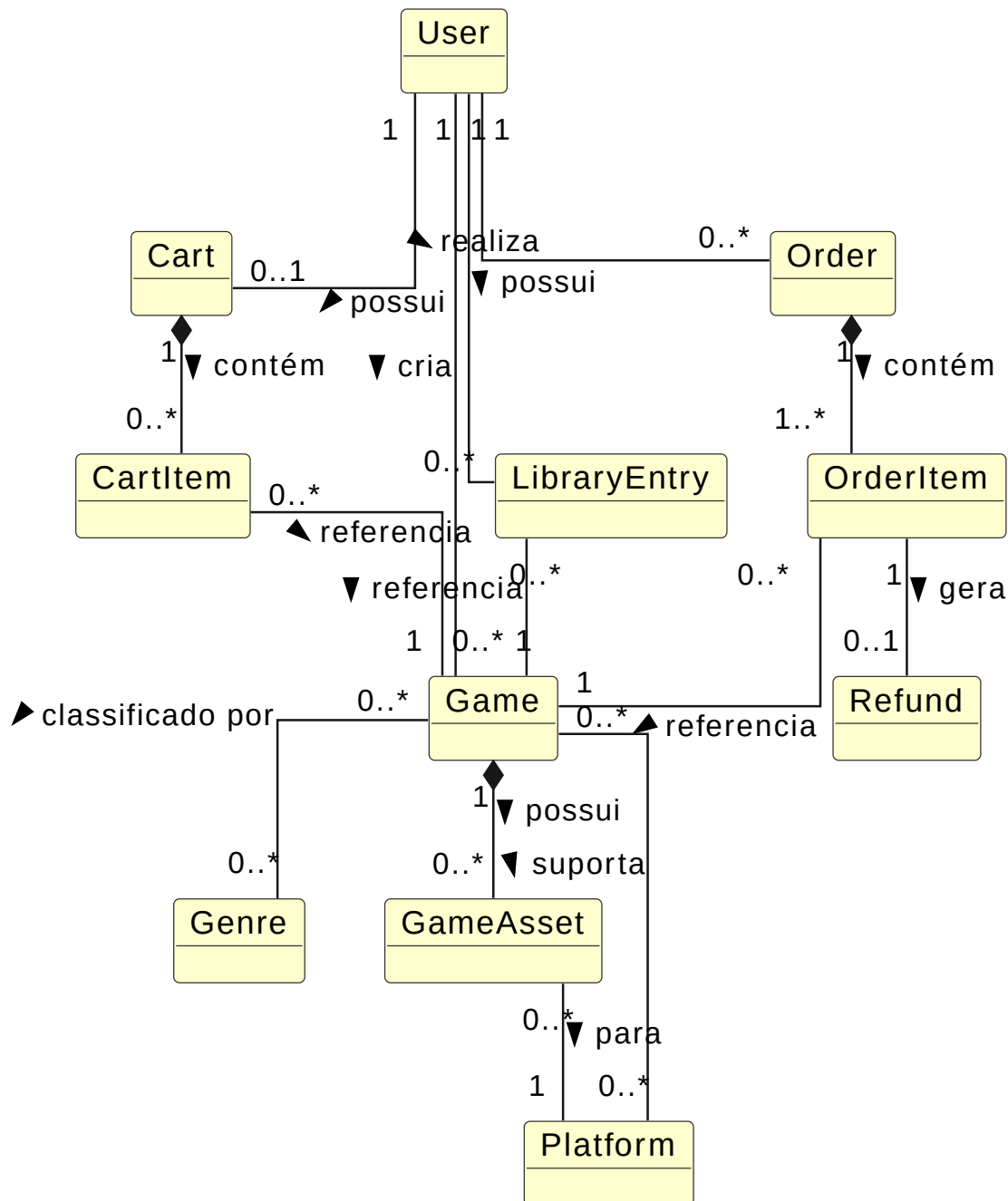


Figura 8. Modelo Conceitual – Vapor

Diagrama Entidade Relacionamento

O diagrama ER detalha as tabelas, colunas, tipos de dados, chaves primárias e chaves estrangeiras do banco de dados relacional.

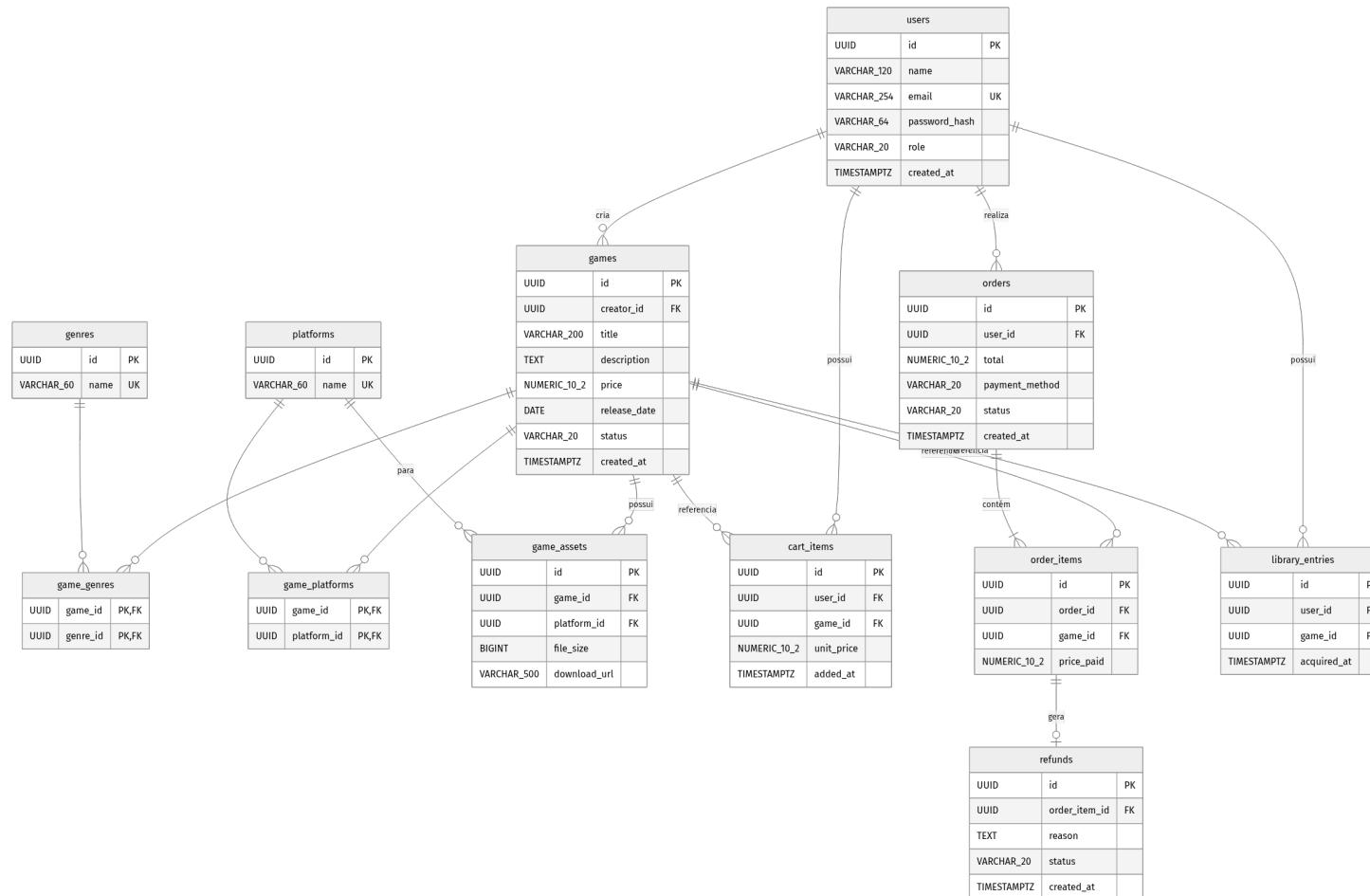


Figura 9. Diagrama ER – Vapor

Diagramas de Classes

Os diagramas estão organizados em cinco domínios de negócio para manter a legibilidade em página A4. Atributos prefixados com # são protegidos.

Domínio 1: Usuários e Autenticação

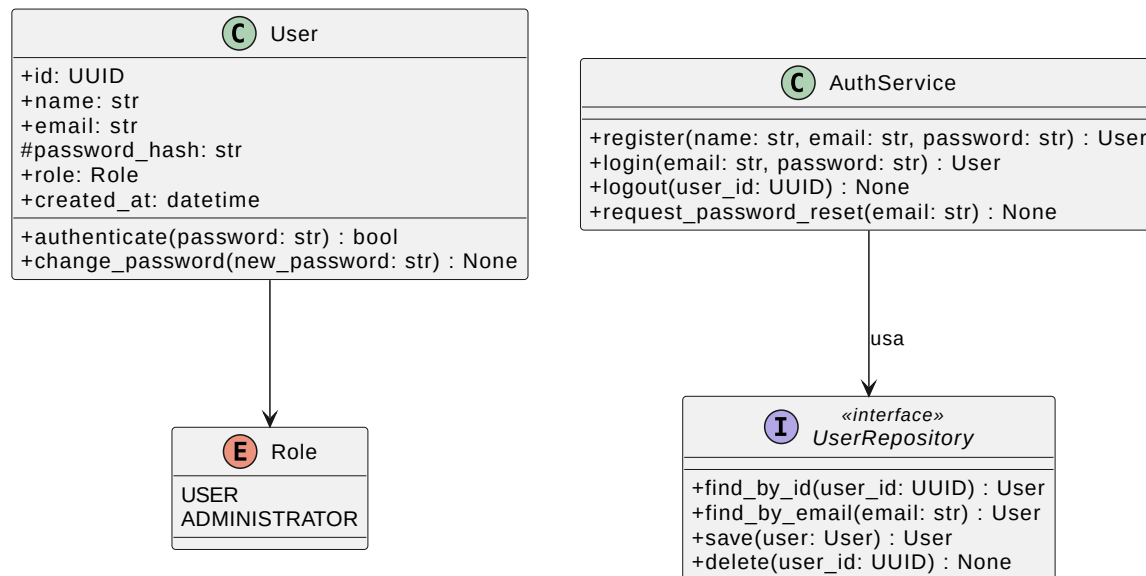


Figura 10. Diagrama de Classes – Usuários e Autenticação

Domínio 2: Catálogo

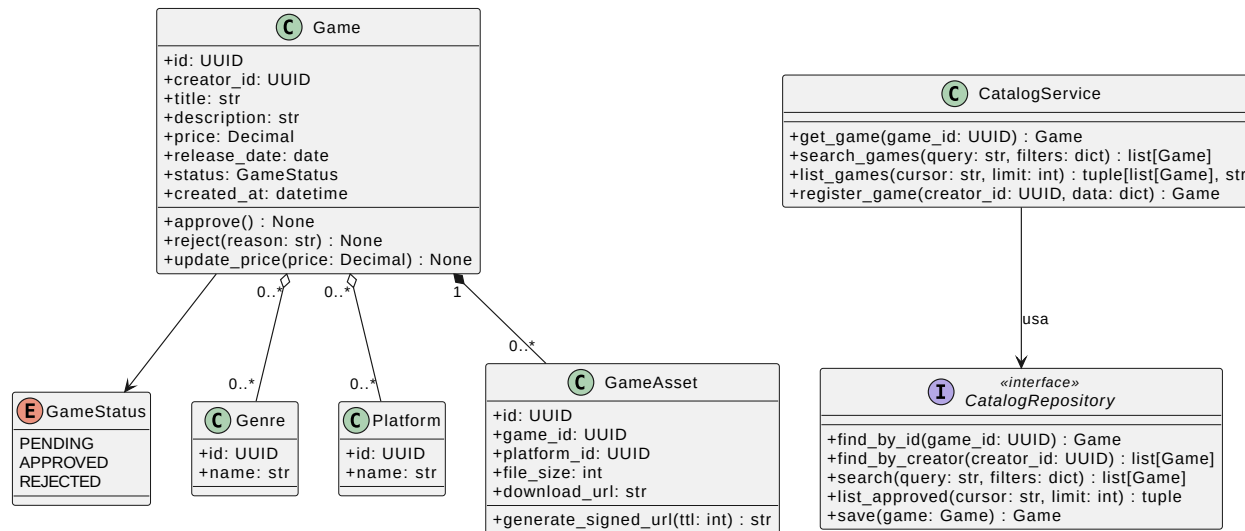


Figura 11. Diagrama de Classes – Catálogo

Domínio 3: Carrinho e Pedidos

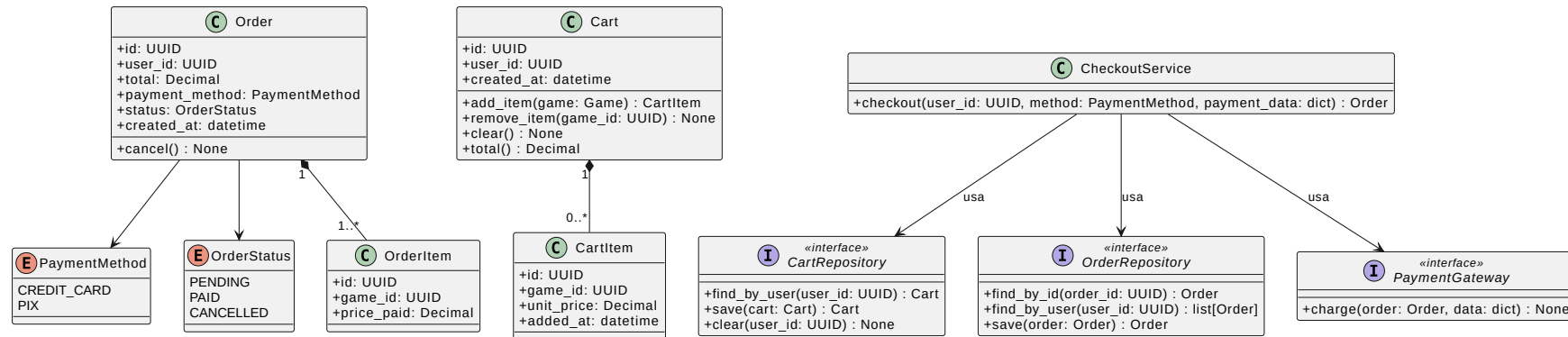


Figura 12. Diagrama de Classes — Carrinho e Pedidos

Domínio 4: Biblioteca, Downloads e Reembolso

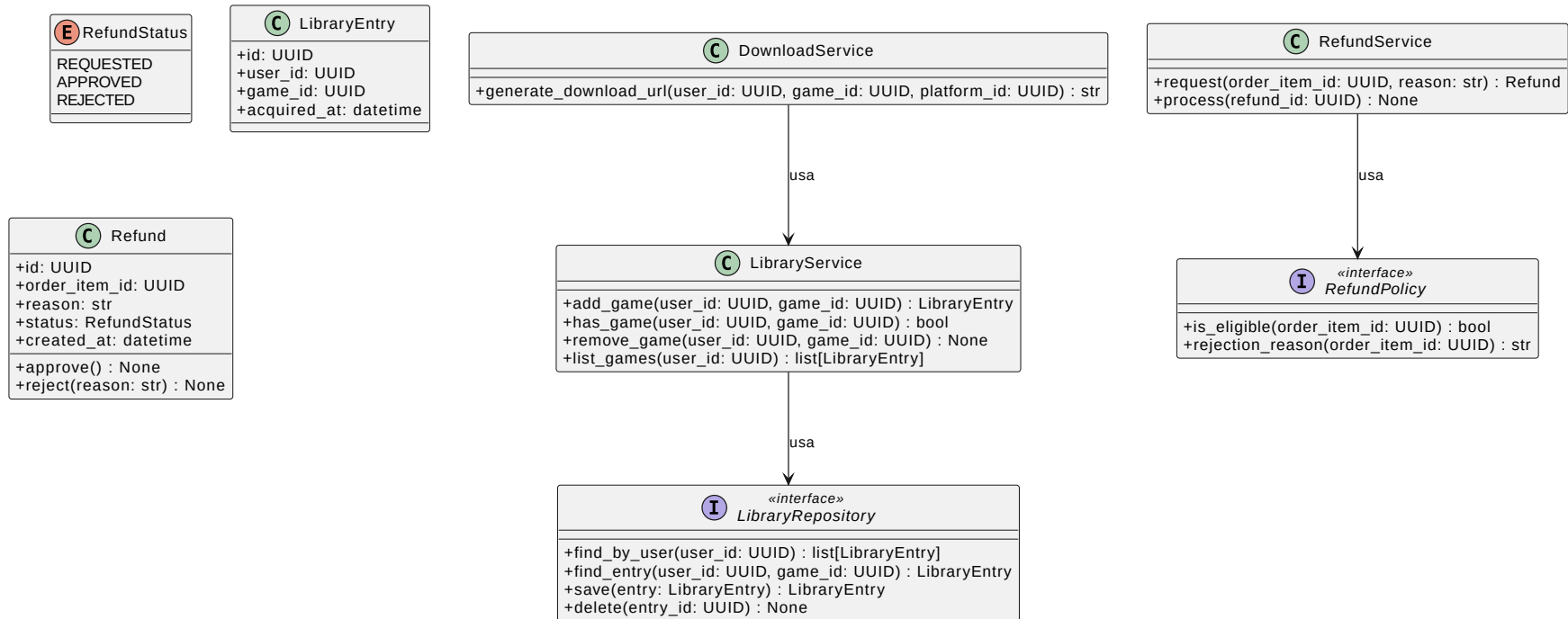


Figura 13. Diagrama de Classes — Biblioteca, Downloads e Reembolso

Domínio 5: Moderação e Pannel de Vendas

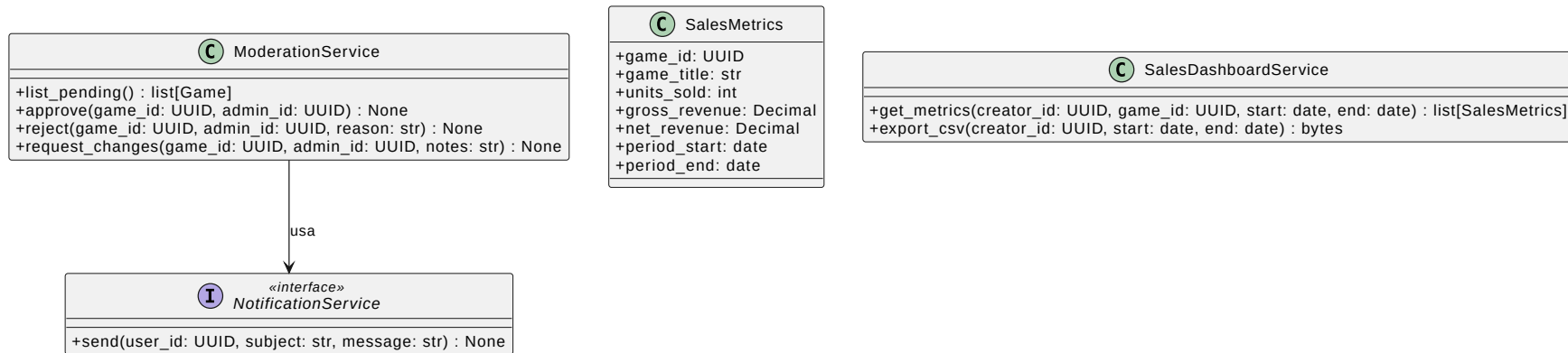


Figura 14. Diagrama de Classes — Moderação e Pannel de Vendas

Codificação das Classes do Sistema

A implementação utiliza Python 3.13 com `dataclasses` para as entidades de domínio e `Enum` para tipos enumerados.

Enumerações

vapor/enums.py

```
from enum import Enum

class Role(Enum):
    USER = 'user'
    ADMINISTRATOR = 'administrator'

class GameStatus(Enum):
    PENDING = 'pending'
    APPROVED = 'approved'
    REJECTED = 'rejected'

class PaymentMethod(Enum):
    CREDIT_CARD = 'credit_card'
    PIX = 'pix'

class OrderStatus(Enum):
    PENDING = 'pending'
    PAID = 'paid'
    CANCELLED = 'cancelled'

class RefundStatus(Enum):
    REQUESTED = 'requested'
    APPROVED = 'approved'
    REJECTED = 'rejected'
```

Domínio 1: Usuários e Autenticação

vapor/users.py

```
from dataclasses import dataclass, field
from datetime import UTC, datetime
from hashlib import sha256
from uuid import UUID, uuid4

from vapor.enums import Role
```

```

@dataclass
class User:
    id: UUID
    name: str
    email: str
    password_hash: str
    role: Role
    created_at: datetime = field(default_factory=lambda: datetime.now(UTC))

    def authenticate(self, password: str) -> bool:
        return self.password_hash == sha256(password.encode()).hexdigest()

    def change_password(self, new_password: str) -> None:
        self.password_hash = sha256(new_password.encode()).hexdigest()

class AuthService:
    def __init__(self, user_repository):
        self._user_repository = user_repository

    def register(self, name: str, email: str, password: str) -> User:
        password_hash = sha256(password.encode()).hexdigest()
        user = User(
            id=uuid4(),
            name=name,
            email=email,
            password_hash=password_hash,
            role=Role.USER,
        )
        return self._user_repository.save(user)

    def login(self, email: str, password: str) -> User:
        user = self._user_repository.find_by_email(email)
        if not user.authenticate(password):
            raise ValueError('Invalid credentials')
        return user

    def logout(self, user_id: UUID) -> None:
        pass

    def request_password_reset(self, email: str) -> None:
        pass

```

Domínio 2: Catálogo

vapor/catalog.py

```

from dataclasses import dataclass, field
from datetime import UTC, date, datetime
from decimal import Decimal
from uuid import UUID, uuid4

```

```

from vapor.enums import GameStatus

@dataclass
class Genre:
    id: UUID
    name: str

@dataclass
class Platform:
    id: UUID
    name: str

@dataclass
class GameAsset:
    id: UUID
    game_id: UUID
    platform_id: UUID
    file_size: int
    download_url: str = ''

    def generate_signed_url(self, ttl: int = 3600) -> str:
        return f'{self.download_url}?token={self.id}&ttl={ttl}'

@dataclass
class Game:
    id: UUID
    creator_id: UUID
    title: str
    description: str
    price: Decimal
    release_date: date
    status: GameStatus
    created_at: datetime = field(default_factory=lambda: datetime.now(UTC))
    genres: list[Genre] = field(default_factory=list)
    platforms: list[Platform] = field(default_factory=list)
    assets: list[GameAsset] = field(default_factory=list)

    def approve(self) -> None:
        self.status = GameStatus.APPROVED

    def reject(self, reason: str) -> None:
        self.status = GameStatus.REJECTED

    def update_price(self, price: Decimal) -> None:
        self.price = price

class CatalogService:
    def __init__(self, catalog_repository):

```

```

self._catalog_repository = catalog_repository

def get_game(self, game_id: UUID) -> Game:
    return self._catalog_repository.find_by_id(game_id)

def search_games(self, query: str, filters: dict) -> list[Game]:
    return self._catalog_repository.search(query, filters)

def list_games(self, cursor: str, limit: int) -> tuple[list[Game], str]:
    return self._catalog_repository.list_approved(cursor, limit)

def register_game(self, creator_id: UUID, data: dict) -> Game:
    game = Game(
        id=uuid4(),
        creator_id=creator_id,
        title=data['title'],
        description=data.get('description', ''),
        price=Decimal(str(data.get('price', '0.00'))),
        release_date=data.get('release_date', date.today()),
        status=GameStatus.PENDING,
    )
    return self._catalog_repository.save(game)

```

Domínio 3: Carrinho e Pedidos

vapor/orders.py

```

from dataclasses import dataclass, field
from datetime import UTC, datetime
from decimal import Decimal
from uuid import UUID, uuid4

from vapor.catalog import Game
from vapor.enums import OrderStatus, PaymentMethod

@dataclass
class CartItem:
    id: UUID
    game_id: UUID
    unit_price: Decimal
    added_at: datetime = field(default_factory=lambda: datetime.now(UTC))

@dataclass
class Cart:
    id: UUID
    user_id: UUID
    created_at: datetime = field(default_factory=lambda: datetime.now(UTC))
    items: list[CartItem] = field(default_factory=list)

    def add_item(self, game: Game) -> CartItem:

```

```

        for item in self.items:
            if item.game_id == game.id:
                raise ValueError('Game already in cart')
            cart_item = CartItem(id=uuid4(), game_id=game.id, unit_price=game.price)
            self.items.append(cart_item)
        return cart_item

    def remove_item(self, game_id: UUID) -> None:
        self.items = [item for item in self.items if item.game_id != game_id]

    def clear(self) -> None:
        self.items.clear()

    def total(self) -> Decimal:
        return sum((item.unit_price for item in self.items), Decimal('0.00'))

@dataclass
class OrderItem:
    id: UUID
    game_id: UUID
    price_paid: Decimal

@dataclass
class Order:
    id: UUID
    user_id: UUID
    total: Decimal
    payment_method: PaymentMethod
    status: OrderStatus
    created_at: datetime = field(default_factory=lambda: datetime.now(UTC))
    items: list[OrderItem] = field(default_factory=list)

    def cancel(self) -> None:
        self.status = OrderStatus.CANCELLED

class CheckoutService:
    def __init__(self, cart_repository, order_repository, payment_gateway,
library_service):
        self._cart_repository = cart_repository
        self._order_repository = order_repository
        self._payment_gateway = payment_gateway
        self._library_service = library_service

    def checkout(self, user_id: UUID, method: PaymentMethod, payment_data: dict) -> Order:
        cart = self._cart_repository.find_by_user(user_id)
        if not cart.items:
            raise ValueError('Cart is empty')

        order_items = [OrderItem(id=uuid4(), game_id=item.game_id, price_paid=item
.unit_price) for item in cart.items]
        order = Order(

```

```

        id=uuid4(),
        user_id=user_id,
        total=cart.total(),
        payment_method=method,
        status=OrderStatus.PENDING,
        items=order_items,
    )

    self._payment_gateway.charge(order, payment_data)
    order.status = OrderStatus.PAID
    self._order_repository.save(order)

    for item in order.items:
        self._library_service.add_game(user_id, item.game_id)

    self._cart_repository.clear(user_id)
    return order

```

Domínio 4: Biblioteca, Downloads e Reembolso

vapor/library.py

```

from dataclasses import dataclass, field
from datetime import UTC, datetime
from uuid import UUID, uuid4

from vapor.enums import RefundStatus

@dataclass
class LibraryEntry:
    id: UUID
    user_id: UUID
    game_id: UUID
    acquired_at: datetime = field(default_factory=lambda: datetime.now(UTC))

@dataclass
class Refund:
    id: UUID
    order_item_id: UUID
    reason: str
    status: RefundStatus
    created_at: datetime = field(default_factory=lambda: datetime.now(UTC))

    def approve(self) -> None:
        self.status = RefundStatus.APPROVED

    def reject(self, reason: str) -> None:
        self.status = RefundStatus.REJECTED

```

```

class LibraryService:
    def __init__(self, library_repository):
        self._library_repository = library_repository

    def add_game(self, user_id: UUID, game_id: UUID) -> LibraryEntry:
        entry = LibraryEntry(id=uuid4(), user_id=user_id, game_id=game_id)
        return self._library_repository.save(entry)

    def has_game(self, user_id: UUID, game_id: UUID) -> bool:
        try:
            self._library_repository.find_entry(user_id, game_id)
            return True
        except LookupError:
            return False

    def remove_game(self, user_id: UUID, game_id: UUID) -> None:
        entry = self._library_repository.find_entry(user_id, game_id)
        self._library_repository.delete(entry.id)

    def list_games(self, user_id: UUID) -> list[LibraryEntry]:
        return self._library_repository.find_by_user(user_id)

class DownloadService:
    def __init__(self, library_service, catalog_repository):
        self._library_service = library_service
        self._catalog_repository = catalog_repository

    def generate_download_url(self, user_id: UUID, game_id: UUID, platform_id: UUID) ->
str:
        if not self._library_service.has_game(user_id, game_id):
            raise PermissionError('Game not in library')
        game = self._catalog_repository.find_by_id(game_id)
        for asset in game.assets:
            if asset.platform_id == platform_id:
                return asset.generate_signed_url()
        raise LookupError('Asset not found for platform')

class RefundService:
    def __init__(self, refund_policy, payment_gateway, library_service):
        self._refund_policy = refund_policy
        self._payment_gateway = payment_gateway
        self._library_service = library_service

    def request(self, order_item_id: UUID, reason: str) -> Refund:
        if not self._refund_policy.is_eligible(order_item_id):
            msg = self._refund_policy.rejection_reason(order_item_id)
            raise ValueError(msg)
        return Refund(
            id=uuid4(),
            order_item_id=order_item_id,
            reason=reason,
            status=RefundStatus.REQUESTED,

```

```
)

def process(self, refund_id: UUID) -> None:
    pass
```

Domínio 5: Moderação e Painel de Vendas

vapor/moderation.py

```
from dataclasses import dataclass
from datetime import date
from decimal import Decimal
from uuid import UUID

from vapor.catalog import Game

@dataclass
class SalesMetrics:
    game_id: UUID
    game_title: str
    units_sold: int
    gross_revenue: Decimal
    net_revenue: Decimal
    period_start: date
    period_end: date

class ModerationService:
    def __init__(self, catalog_repository, notification_service):
        self._catalog_repository = catalog_repository
        self._notification_service = notification_service

    def list_pending(self) -> list[Game]:
        results, _ = self._catalog_repository.search('', {'status': 'pending'})
        return results

    def approve(self, game_id: UUID, admin_id: UUID) -> None:
        game = self._catalog_repository.find_by_id(game_id)
        game.approve()
        self._catalog_repository.save(game)
        self._notification_service.send(
            game.creator_id,
            'Game approved',
            f'Your game "{game.title}" has been approved and is now available in the
catalog.',
        )

    def reject(self, game_id: UUID, admin_id: UUID, reason: str) -> None:
        game = self._catalog_repository.find_by_id(game_id)
        game.reject(reason)
        self._catalog_repository.save(game)
```

```

        self._notification_service.send(
            game.creator_id,
            'Game rejected',
            f'Your game "{game.title}" has been rejected. Reason: {reason}',
        )

    def request_changes(self, game_id: UUID, admin_id: UUID, notes: str) -> None:
        game = self._catalog_repository.find_by_id(game_id)
        self._notification_service.send(
            game.creator_id,
            'Changes requested',
            f'Your game "{game.title}" needs changes: {notes}',
        )

class SalesDashboardService:
    def __init__(self, order_repository):
        self._order_repository = order_repository

    def get_metrics(self, creator_id: UUID, game_id: UUID, start: date, end: date) -> list[SalesMetrics]:
        return []

    def export_csv(self, creator_id: UUID, start: date, end: date) -> bytes:
        return b''

```

Codificação do Acesso ao Banco de Dados

O acesso ao banco de dados utiliza PostgreSQL 17 via `psycopg` v3. O arquivo `schema.sql` define o esquema relacional (DDL) e o módulo `database.py` gerencia a conexão e a inicialização do banco. O módulo `repositories.py` implementa os repositórios de acesso a dados, utilizando SQL direto com *prepared statements* parametrizados (`%s`) e `ON CONFLICT` para operações de *upsert*.

Esquema do Banco de Dados

`schema.sql`

```

CREATE TABLE IF NOT EXISTS users (
    id UUID PRIMARY KEY,
    name VARCHAR(120) NOT NULL,
    email VARCHAR(254) NOT NULL UNIQUE,
    password_hash VARCHAR(64) NOT NULL,
    role VARCHAR(20) NOT NULL CHECK (role IN ('user', 'administrator')),
    created_at TIMESTAMPTZ NOT NULL
);

CREATE TABLE IF NOT EXISTS games (
    id UUID PRIMARY KEY,
    creator_id UUID NOT NULL REFERENCES users(id),
    title VARCHAR(200) NOT NULL,

```

```

        description TEXT DEFAULT '',
        price NUMERIC(10,2) NOT NULL,
        release_date DATE,
        status VARCHAR(20) NOT NULL CHECK (status IN ('pending', 'approved', 'rejected')),
        created_at TIMESTAMPTZ NOT NULL
    );

CREATE TABLE IF NOT EXISTS genres (
    id UUID PRIMARY KEY,
    name VARCHAR(60) NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS platforms (
    id UUID PRIMARY KEY,
    name VARCHAR(60) NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS game_genres (
    game_id UUID NOT NULL REFERENCES games(id),
    genre_id UUID NOT NULL REFERENCES genres(id),
    PRIMARY KEY (game_id, genre_id)
);

CREATE TABLE IF NOT EXISTS game_platforms (
    game_id UUID NOT NULL REFERENCES games(id),
    platform_id UUID NOT NULL REFERENCES platforms(id),
    PRIMARY KEY (game_id, platform_id)
);

CREATE TABLE IF NOT EXISTS game_assets (
    id UUID PRIMARY KEY,
    game_id UUID NOT NULL REFERENCES games(id),
    platform_id UUID NOT NULL REFERENCES platforms(id),
    file_size BIGINT NOT NULL,
    download_url VARCHAR(500) DEFAULT ''
);

CREATE TABLE IF NOT EXISTS cart_items (
    id UUID PRIMARY KEY,
    user_id UUID NOT NULL REFERENCES users(id),
    game_id UUID NOT NULL REFERENCES games(id),
    unit_price NUMERIC(10,2) NOT NULL,
    added_at TIMESTAMPTZ NOT NULL
);

CREATE TABLE IF NOT EXISTS orders (
    id UUID PRIMARY KEY,
    user_id UUID NOT NULL REFERENCES users(id),
    total NUMERIC(10,2) NOT NULL,
    payment_method VARCHAR(20) NOT NULL CHECK (payment_method IN ('credit_card', 'pix')),
    status VARCHAR(20) NOT NULL CHECK (status IN ('pending', 'paid', 'cancelled')),
    created_at TIMESTAMPTZ NOT NULL
);

```

```

CREATE TABLE IF NOT EXISTS order_items (
    id UUID PRIMARY KEY,
    order_id UUID NOT NULL REFERENCES orders(id),
    game_id UUID NOT NULL REFERENCES games(id),
    price_paid NUMERIC(10,2) NOT NULL
);

CREATE TABLE IF NOT EXISTS library_entries (
    id UUID PRIMARY KEY,
    user_id UUID NOT NULL REFERENCES users(id),
    game_id UUID NOT NULL REFERENCES games(id),
    acquired_at TIMESTAMPTZ NOT NULL,
    UNIQUE (user_id, game_id)
);

CREATE TABLE IF NOT EXISTS refunds (
    id UUID PRIMARY KEY,
    order_item_id UUID NOT NULL REFERENCES order_items(id),
    reason TEXT NOT NULL,
    status VARCHAR(20) NOT NULL CHECK (status IN ('requested', 'approved', 'rejected')),
    created_at TIMESTAMPTZ NOT NULL
);

```

Conexão com o Banco de Dados

vapor/database.py

```

import os
from pathlib import Path

import psycopg
from psycopg.rows import dict_row

DATABASE_URL = os.environ.get('DATABASE_URL',
    'postgresql://vapor:vapor@localhost:5432/vapor')

SCHEMA_PATH = Path(__file__).resolve().parent.parent / 'schema.sql'

def get_connection(dsn=None):
    return psycopg.connect(dsn or DATABASE_URL, row_factory=dict_row)

def init_db(dsn=None):
    conn = get_connection(dsn)
    conn.execute(SCHEMA_PATH.read_text())
    conn.commit()
    return conn

```

Implementação dos Repositórios

vapor/repositories.py

```
from decimal import Decimal
from uuid import UUID, uuid4

from vapor.catalog import Game, GameAsset, Genre, Platform
from vapor.database import get_connection
from vapor.enums import GameStatus, OrderStatus, PaymentMethod, RefundStatus, Role
from vapor.library import LibraryEntry, Refund
from vapor.orders import Cart, CartItem, Order, OrderItem
from vapor.users import User

class PgUserRepository:
    def __init__(self, dsn=None):
        self._dsn = dsn

    def find_by_id(self, user_id: UUID) -> User:
        conn = get_connection(self._dsn)
        row = conn.execute('SELECT * FROM users WHERE id = %s', (user_id,)).fetchone()
        conn.close()
        if not row:
            raise LookupError(f'User {user_id} not found')
        return self._row_to_user(row)

    def find_by_email(self, email: str) -> User:
        conn = get_connection(self._dsn)
        row = conn.execute('SELECT * FROM users WHERE email = %s', (email,)).fetchone()
        conn.close()
        if not row:
            raise LookupError(f'User with email {email} not found')
        return self._row_to_user(row)

    def save(self, user: User) -> User:
        conn = get_connection(self._dsn)
        conn.execute(
            'INSERT INTO users (id, name, email, password_hash, role, created_at)'
            ' VALUES (%s, %s, %s, %s, %s, %s)'
            ' ON CONFLICT (id) DO UPDATE SET name=EXCLUDED.name, email=EXCLUDED.email,'
            ' password_hash=EXCLUDED.password_hash, role=EXCLUDED.role',
            (user.id, user.name, user.email, user.password_hash, user.role.value, user
            .created_at),
        )
        conn.commit()
        conn.close()
        return user

    def delete(self, user_id: UUID) -> None:
        conn = get_connection(self._dsn)
        conn.execute('DELETE FROM users WHERE id = %s', (user_id,))
        conn.commit()
```

```

conn.close()

def _row_to_user(self, row) -> User:
    return User(
        id=row['id'],
        name=row['name'],
        email=row['email'],
        password_hash=row['password_hash'],
        role=Role(row['role']),
        created_at=row['created_at'],
    )

class PgCatalogRepository:
    def __init__(self, dsn=None):
        self._dsn = dsn

    def find_by_id(self, game_id: UUID) -> Game:
        conn = get_connection(self._dsn)
        row = conn.execute('SELECT * FROM games WHERE id = %s', (game_id,)).fetchone()
        if not row:
            conn.close()
            raise LookupError(f'Game {game_id} not found')
        game = self._row_to_game(conn, row)
        conn.close()
        return game

    def find_by_creator(self, creator_id: UUID) -> list[Game]:
        conn = get_connection(self._dsn)
        rows = conn.execute('SELECT * FROM games WHERE creator_id = %s', (creator_id
,)).fetchall()
        games = [self._row_to_game(conn, row) for row in rows]
        conn.close()
        return games

    def search(self, query: str, filters: dict) -> list[Game]:
        conn = get_connection(self._dsn)
        sql = 'SELECT * FROM games WHERE true'
        params: list = []
        if query:
            sql += ' AND title ILIKE %s'
            params.append(f'%{query}%')
        if 'status' in filters:
            sql += ' AND status = %s'
            params.append(filters['status'])
        if 'genre' in filters:
            sql += (
                ' AND id IN (SELECT game_id FROM game_genres'
                ' JOIN genres ON genre_id = genres.id WHERE genres.name = %s)'
            )
            params.append(filters['genre'])
        rows = conn.execute(sql, params).fetchall()
        games = [self._row_to_game(conn, row) for row in rows]
        conn.close()

```

```

        return games

    def list_approved(self, cursor: str, limit: int) -> tuple[list[Game], str]:
        conn = get_connection(self._dsn)
        sql = "SELECT * FROM games WHERE status = 'approved'"
        params: list = []
        if cursor:
            sql += ' AND created_at > %s'
            params.append(cursor)
        sql += ' ORDER BY created_at ASC LIMIT %s'
        params.append(limit)
        rows = conn.execute(sql, params).fetchall()
        games = [self._row_to_game(conn, row) for row in rows]
        conn.close()
        next_cursor = games[-1].created_at.isoformat() if games else ''
        return games, next_cursor

    def save(self, game: Game) -> Game:
        conn = get_connection(self._dsn)
        conn.execute(
            'INSERT INTO games (id, creator_id, title, description, price, release_date,
status, created_at)'
            ' VALUES (%s, %s, %s, %s, %s, %s, %s, %s)'
            ' ON CONFLICT (id) DO UPDATE SET title=EXCLUDED.title,
description=EXCLUDED.description,'
            ' price=EXCLUDED.price, release_date=EXCLUDED.release_date,
status=EXCLUDED.status',
            (
                game.id,
                game.creator_id,
                game.title,
                game.description,
                game.price,
                game.release_date,
                game.status.value,
                game.created_at,
            ),
        )
        conn.execute('DELETE FROM game_genres WHERE game_id = %s', (game.id,))
        for genre in game.genres:
            existing = conn.execute('SELECT id FROM genres WHERE name = %s', (genre.
name,)).fetchone()
            if existing:
                genre_id = existing['id']
            else:
                genre_id = genre.id
                conn.execute('INSERT INTO genres (id, name) VALUES (%s, %s)', (genre_id,
genre.name))
            conn.execute('INSERT INTO game_genres (game_id, genre_id) VALUES (%s, %s)',
(game.id, genre_id))
        conn.execute('DELETE FROM game_platforms WHERE game_id = %s', (game.id,))
        for platform in game.platforms:
            existing = conn.execute('SELECT id FROM platforms WHERE name = %s', (platform
.name,)).fetchone()

```

```

        if existing:
            platform_id = existing['id']
        else:
            platform_id = platform.id
            conn.execute('INSERT INTO platforms (id, name) VALUES (%s, %s)',
(platform_id, platform.name))
            conn.execute(
                'INSERT INTO game_platforms (game_id, platform_id) VALUES (%s, %s)',
                (game.id, platform_id),
            )
        conn.commit()
        conn.close()
        return game

    def _row_to_game(self, conn, row) -> Game:
        game_id = row['id']
        genre_rows = conn.execute(
            'SELECT g.* FROM genres g JOIN game_genres gg ON g.id = gg.genre_id WHERE
gg.game_id = %s', (game_id,)
        ).fetchall()
        platform_rows = conn.execute(
            'SELECT p.* FROM platforms p JOIN game_platforms gp ON p.id = gp.platform_id
WHERE gp.game_id = %s',
            (game_id,),
        ).fetchall()
        asset_rows = conn.execute('SELECT * FROM game_assets WHERE game_id = %s', (
game_id,)).fetchall()
        return Game(
            id=game_id,
            creator_id=row['creator_id'],
            title=row['title'],
            description=row['description'] or '',
            price=Decimal(row['price']),
            release_date=row['release_date'],
            status=GameStatus(row['status']),
            created_at=row['created_at'],
            genres=[Genre(id=r['id'], name=r['name']) for r in genre_rows],
            platforms=[Platform(id=r['id'], name=r['name']) for r in platform_rows],
            assets=[
                GameAsset(
                    id=r['id'],
                    game_id=r['game_id'],
                    platform_id=r['platform_id'],
                    file_size=r['file_size'],
                    download_url=r['download_url'] or '',
                )
                for r in asset_rows
            ],
        )

class PgCartRepository:
    def __init__(self, dsn=None):
        self._dsn = dsn

```

```

def find_by_user(self, user_id: UUID) -> Cart:
    conn = get_connection(self._dsn)
    rows = conn.execute('SELECT * FROM cart_items WHERE user_id = %s ORDER BY
added_at', (user_id,)).fetchall()
    conn.close()
    items = [
        CartItem(
            id=r['id'],
            game_id=r['game_id'],
            unit_price=Decimal(r['unit_price']),
            added_at=r['added_at'],
        )
        for r in rows
    ]
    return Cart(id=uuid4(), user_id=user_id, items=items)

def save(self, cart: Cart) -> Cart:
    conn = get_connection(self._dsn)
    conn.execute('DELETE FROM cart_items WHERE user_id = %s', (cart.user_id,))
    for item in cart.items:
        conn.execute(
            'INSERT INTO cart_items (id, user_id, game_id, unit_price, added_at)
VALUES (%s, %s, %s, %s, %s)',
            (item.id, cart.user_id, item.game_id, item.unit_price, item.added_at),
        )
    conn.commit()
    conn.close()
    return cart

def clear(self, user_id: UUID) -> None:
    conn = get_connection(self._dsn)
    conn.execute('DELETE FROM cart_items WHERE user_id = %s', (user_id,))
    conn.commit()
    conn.close()

class PgOrderRepository:
    def __init__(self, dsn=None):
        self._dsn = dsn

    def find_by_id(self, order_id: UUID) -> Order:
        conn = get_connection(self._dsn)
        row = conn.execute('SELECT * FROM orders WHERE id = %s', (order_id,)).fetchone()
        if not row:
            conn.close()
            raise LookupError(f'Order {order_id} not found')
        order = self._row_to_order(conn, row)
        conn.close()
        return order

    def find_by_user(self, user_id: UUID) -> list[Order]:
        conn = get_connection(self._dsn)
        rows = conn.execute('SELECT * FROM orders WHERE user_id = %s ORDER BY created_at

```

```

DESC', (user_id,)).fetchall()
    orders = [self._row_to_order(conn, row) for row in rows]
    conn.close()
    return orders

def save(self, order: Order) -> Order:
    conn = get_connection(self._dsn)
    conn.execute(
        'INSERT INTO orders (id, user_id, total, payment_method, status, created_at)'
        ' VALUES (%s, %s, %s, %s, %s, %s)'
        ' ON CONFLICT (id) DO UPDATE SET status=EXCLUDED.status',
        (order.id, order.user_id, order.total, order.payment_method.value, order
.status.value, order.created_at),
    )
    for item in order.items:
        conn.execute(
            'INSERT INTO order_items (id, order_id, game_id, price_paid) VALUES (%s,
%s, %s, %s)'
            ' ON CONFLICT (id) DO NOTHING',
            (item.id, order.id, item.game_id, item.price_paid),
        )
    conn.commit()
    conn.close()
    return order

def _row_to_order(self, conn, row) -> Order:
    item_rows = conn.execute('SELECT * FROM order_items WHERE order_id = %s', (row['
id'],)).fetchall()
    return Order(
        id=row['id'],
        user_id=row['user_id'],
        total=Decimal(row['total']),
        payment_method=PaymentMethod(row['payment_method']),
        status=OrderStatus(row['status']),
        created_at=row['created_at'],
        items=[OrderItem(id=r['id'], game_id=r['game_id'], price_paid=Decimal(r[
'price_paid'])) for r in item_rows],
    )

class PgLibraryRepository:
    def __init__(self, dsn=None):
        self._dsn = dsn

    def find_by_user(self, user_id: UUID) -> list[LibraryEntry]:
        conn = get_connection(self._dsn)
        rows = conn.execute(
            'SELECT * FROM library_entries WHERE user_id = %s ORDER BY acquired_at',
            (user_id,)
        ).fetchall()
        conn.close()
        return [self._row_to_entry(r) for r in rows]

    def find_entry(self, user_id: UUID, game_id: UUID) -> LibraryEntry:

```

```

        conn = get_connection(self._dsn)
        row = conn.execute(
            'SELECT * FROM library_entries WHERE user_id = %s AND game_id = %s', (user_id,
game_id)
        ).fetchone()
        conn.close()
        if not row:
            raise LookupError(f'Library entry not found for user={user_id} game={game_id}')
        return self._row_to_entry(row)

    def save(self, entry: LibraryEntry) -> LibraryEntry:
        conn = get_connection(self._dsn)
        conn.execute(
            'INSERT INTO library_entries (id, user_id, game_id, acquired_at) VALUES (%s,
%s, %s, %s)'
            ' ON CONFLICT (id) DO NOTHING',
            (entry.id, entry.user_id, entry.game_id, entry.acquired_at),
        )
        conn.commit()
        conn.close()
        return entry

    def delete(self, entry_id: UUID) -> None:
        conn = get_connection(self._dsn)
        conn.execute('DELETE FROM library_entries WHERE id = %s', (entry_id,))
        conn.commit()
        conn.close()

    def _row_to_entry(self, row) -> LibraryEntry:
        return LibraryEntry(
            id=row['id'],
            user_id=row['user_id'],
            game_id=row['game_id'],
            acquired_at=row['acquired_at'],
        )

class PgRefundRepository:
    def __init__(self, dsn=None):
        self._dsn = dsn

    def find_by_id(self, refund_id: UUID) -> Refund:
        conn = get_connection(self._dsn)
        row = conn.execute('SELECT * FROM refunds WHERE id = %s', (refund_id,)).fetchone()
        conn.close()
        if not row:
            raise LookupError(f'Refund {refund_id} not found')
        return self._row_to_refund(row)

    def save(self, refund: Refund) -> Refund:
        conn = get_connection(self._dsn)
        conn.execute(
            'INSERT INTO refunds (id, order_item_id, reason, status, created_at) VALUES

```

```

(%s, %s, %s, %s, %s)'
    ' ON CONFLICT (id) DO UPDATE SET status=EXCLUDED.status',
    (refund.id, refund.order_item_id, refund.reason, refund.status.value, refund
.created_at),
    )
    conn.commit()
    conn.close()
    return refund

def _row_to_refund(self, row) -> Refund:
    return Refund(
        id=row['id'],
        order_item_id=row['order_item_id'],
        reason=row['reason'],
        status=RefundStatus(row['status']),
        created_at=row['created_at'],
    )

```